

BÀI THỰC HÀNH: CẤU TRÚC DỮ LIỆU ĐỒ THỊ (GRAPH) - Phần2

1. MỤC TIÊU BÀI THỰC HÀNH

- Biết các cách cài đặt thuật toán tìm đường đi ngắn nhất từ một đỉnh đến các đỉnh còn lại bằng thuật toán Dijkstra bằng ngôn ngữ lập trình Java
- Biết các cách cài đặt thuật toán tìm đường đi ngắn nhất giữa các cặp đỉnh bằng thuật toán Dijkstra bằng ngôn ngữ lập trình Java
- Ứng dụng đồ thị để giải các bài toán thực tế như tìm đường đi, mạng xã hội, sơ đồ luồng công việc.

2. CÁC VÍ DỤ MINH HỌA (CHƯƠNG TRÌNH HOÀN CHỈNH)

Ví dụ 1: Cài đặt thuật toán Dijkstra sử dụng danh sách các đỉnh kề

```
import java.util.*;

class Edge {
    int to, weight;

    Edge(int to, int weight) { this.to = to; this.weight = weight; }
}

class Node implements Comparable<Node> {
    int vertex;
    long dist;

    Node(int vertex, long dist) { this.vertex = vertex; this.dist = dist; }

    public int compareTo(Node o) { return Long.compare(this.dist, o.dist); }
}

class Result {
    long[] dist;
    int[] prev;

    Result(long[] dist, int[] prev) { this.dist = dist; this.prev = prev; }
```

```

    }

    public class DijkstraAllPaths {

        private final int n;

        private final List<List<Edge>> adj;

        public DijkstraAllPaths(int n) {

            this.n = n;

            this.adj = new ArrayList<>();

            for (int i = 0; i < n; i++) adj.add(new ArrayList<>());

        }

        public void addEdge(int u, int v, int w) {

            adj.get(u).add(new Edge(v, w));

        }

        public Result dijkstra(int source) {

            final long INF = Long.MAX_VALUE / 4;

            long[] dist = new long[n];

            int[] prev = new int[n];

            Arrays.fill(dist, INF);

            Arrays.fill(prev, -1);

            PriorityQueue<Node> pq = new PriorityQueue<>();

            boolean[] visited = new boolean[n];

            dist[source] = 0;

            pq.add(new Node(source, 0));

            while (!pq.isEmpty()) {

                Node cur = pq.poll();

                int u = cur.vertex;

                if (visited[u]) continue;

```

```

        visited[u] = true;

        for (Edge e : adj.get(u)) {

            int v = e.to;

            long nd = dist[u] + e.weight;

            if (nd < dist[v]) {

                dist[v] = nd;

                prev[v] = u;

                pq.add(new Node(v, nd));

            }

        }

    }

    return new Result(dist, prev);

}

public List<Integer> getPath(int[] prev, int source, int target) {

    List<Integer> path = new ArrayList<>();

    for (int at = target; at != -1; at = prev[at]) path.add(at);

    Collections.reverse(path);

    if (path.isEmpty() || path.get(0) != source) return Collections.emptyList();

    return path;

}

public void printAllShortestPaths(int source) {

    Result res = dijkstra(source);

    long[] dist = res.dist;

    int[] prev = res.prev;

    System.out.println("=== DUONG DI NGAN NHAT TU DINH " + source + " ===");

    for (int v = 0; v < n; v++) {

```

```

        if (dist[v] >= Long.MAX_VALUE / 8) {

            System.out.println("Đỉnh " + v + ": KHONG CO DUONG DI");

        } else {

            List<Integer> path = getPath(prev, source, v);

            System.out.println("Dinh" + v + " | Khoang cach= " + dist[v] + " | Duong di= " +
path);

        }

    }

}

public static void main(String[] args) {

    DijkstraAllPaths solver = new DijkstraAllPaths(6);

    solver.addEdge(0, 1, 7);

    solver.addEdge(0, 2, 9);

    solver.addEdge(0, 5, 14);

    solver.addEdge(1, 2, 10);

    solver.addEdge(1, 3, 15);

    solver.addEdge(2, 3, 11);

    solver.addEdge(2, 5, 2);

    solver.addEdge(3, 4, 6);

    solver.addEdge(4, 5, 9);

    int source = 0;

    solver.printAllShortestPaths(source);

}

}

```

Ví dụ 2: Cài đặt đồ thị bằng ma trận kề và thuật toán Floyd tìm đường đi ngắn nhất giữa mọi cặp đỉnh

```
import java.util.*;

public class FloydWarshall {

    private static final long INF = Long.MAX_VALUE / 4;

    private final int n;

    private final long[][] dist;

    private final int[][] next;

    public FloydWarshall(int n) {

        this.n = n;

        dist = new long[n][n];

        next = new int[n][n];

        // Khởi tạo

        for (int i = 0; i < n; i++) {

            Arrays.fill(dist[i], INF);

            dist[i][i] = 0;

            Arrays.fill(next[i], -1);

        }

    }

    public void addEdge(int u, int v, long w) {

        dist[u][v] = w;

        next[u][v] = v;

    }

}
```

```
// Thuật toán Floyd-Warshall
```

```
public void run() {  
    for (int k = 0; k < n; k++) {  
        for (int i = 0; i < n; i++) {  
            if (dist[i][k] == INF) continue;  
            for (int j = 0; j < n; j++) {  
                if (dist[k][j] == INF) continue;  
                long newDist = dist[i][k] + dist[k][j];  
                if (newDist < dist[i][j]) {  
                    dist[i][j] = newDist;  
                    next[i][j] = next[i][k];  
                }  
            }  
        }  
    }  
}
```

```
// Khôi phục đường đi giữa u và v
```

```
public List<Integer> getPath(int u, int v) {  
    if (next[u][v] == -1) return Collections.emptyList();  
    List<Integer> path = new ArrayList<>();  
    int at = u;  
    while (at != v) {  
        path.add(at);  
        at = next[at][v];  
    }  
    path.add(v);  
}
```

```

        return path;
    }

    // In bảng kết quả
    public void printAllPairs() {
        System.out.println("=== KẾT QUẢ FLOYD-WARSHALL ===");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print("Từ " + i + " đến " + j + ": ");
                if (dist[i][j] == INF) {
                    System.out.println("Không có đường đi");
                } else {
                    System.out.println("Khoảng cách = " + dist[i][j]
                        + " | Đường đi = " + getPath(i, j));
                }
            }
        }
    }

    // Ví dụ chạy
    public static void main(String[] args) {
        FloydWarshall fw = new FloydWarshall(4);
        fw.addEdge(0, 1, 3);
        fw.addEdge(0, 2, 8);
        fw.addEdge(1, 2, 2);
        fw.addEdge(2, 3, 1);
        fw.addEdge(0, 3, 10);
        fw.run();
    }

```

```
        fw.printAllPairs();  
    }  
}
```

3. NỘI DUNG BÀI THỰC HÀNH

Bài 1: Cài đặt thuật toán Dijkstra sử dụng danh sách các đỉnh kề

Bài 2: Ứng dụng đồ thị giải bài toán xếp hạng mức độ quan trọng của website
PageRank (Google search)